

```
Activities Terminal May 3 11:32 nivass@localhost:~ — ./pc
[nivass@localhost ~]$ ls
'\'
```

a	c	file.txt	marks.txt	reader	sec.awk	third.sh
ab	data.txt	first.csv	Music	reader.c	second.sh	val.awk
ab.awkl	Desktop	fork	opendir	rec	sender	Videos
ab.c	Documents	fork.c	opendir.c	receiver	sender.c	vj.awk
a.out	Downloads	four.sh	pari	receiver.c	shell	write
av.awk	emp.awk	frogprog.c	Pictures	reversenum	shellscrp	writer
awk	emp.txt	frokprog.c	program	rev.sh	shell.sh	writer.c
b	fibo.sh	id.awk	program.c	script.awk	sixth.sh	
	fifth.sh	marks.awk	Public	scripts	Templates	

```
[nivass@localhost ~]$ vi pc.c
[nivass@localhost ~]$ gcc pc.c -o pc -lpthread
[nivass@localhost ~]$ ./pc
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Produced: 6
Consumed: 4
Produced: 7
Produced: 8
Consumed: 5
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
```

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full, mutex;

void* producer(void* arg) {
    int item = 1;
    while(item <= 10) {
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        sem_post(&mutex);
        sem_post(&full);

        item++;
        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg) {
    int item;
    while(1) {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;
    }
}

```

ActivitiesTerminalMay 3 11:34nivass@localhost:~ — /usr/bin/vim pc.c

```
    }
    return NULL;
}

void* consumer(void* arg) {
    int item;
    while(1) {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        sem_post(&mutex);
        sem_post(&empty);

        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t p;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&g, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(g, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}
```

68,1Bot